

DJ Mixing Station Studio - Developer Step-by-Step Guide (offline-v2)

1. Purpose

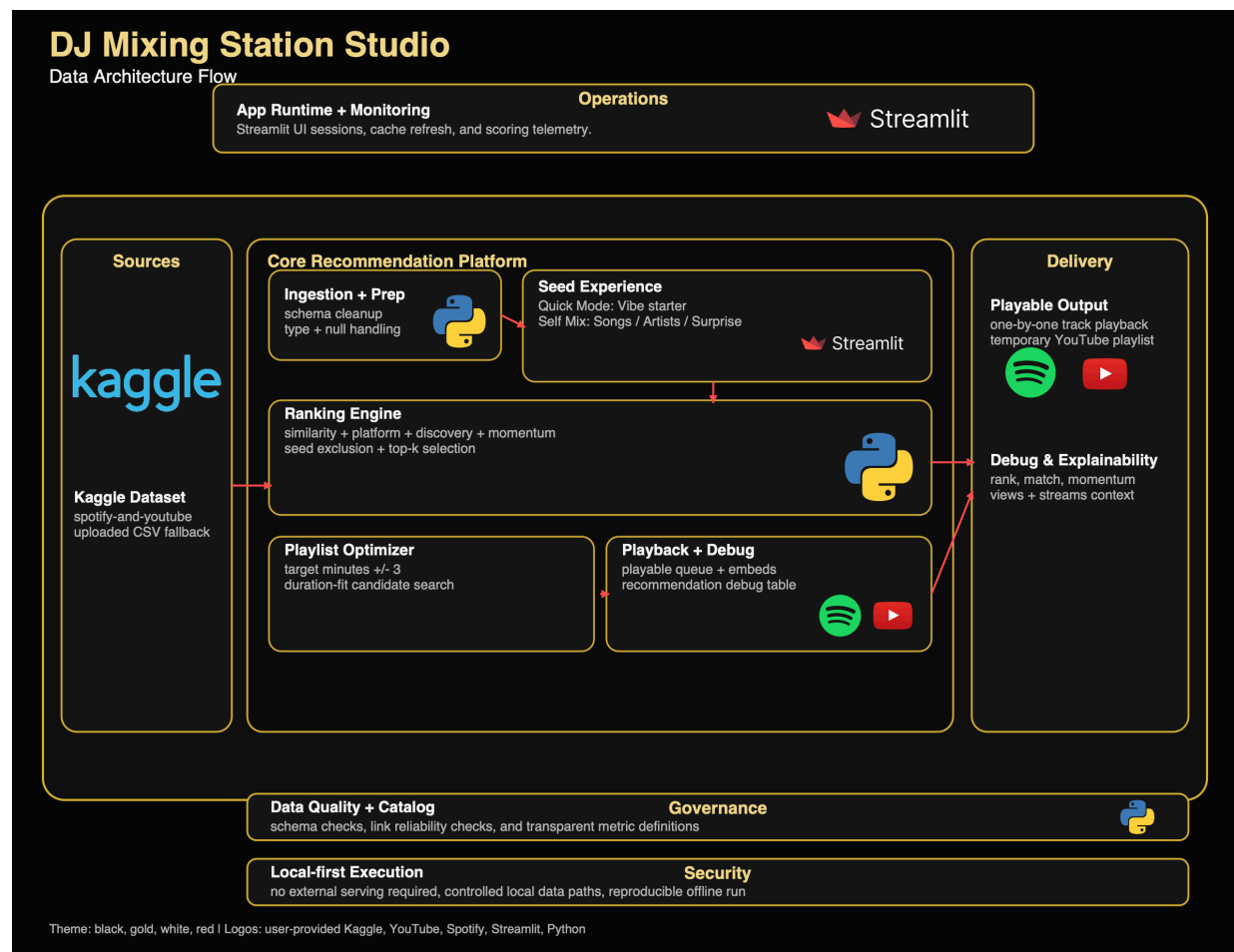
This document explains how the recommender is built end-to-end, including: - system flow - feature definitions and equations - scoring and ranking equations - playlist optimization logic - metrics shown in the UI/debug view - guardrails and edge-case handling

Use this as the main technical implementation reference.

offline-v2 update summary

- Data source is now a dual-Kaggle merge (baseline + expansion).
- App entrypoint is `application_v2.py` (not legacy `app.py`).
- Resolver cache/quota diagnostics and debug-table cache columns are included.
- Link-based dedupe is applied in ranked candidates and playlist construction.

2. Architecture Flow



Data Architecture Flow

Core pipeline: 1. Source data load (dual KaggleHub merge or uploaded CSV) 2. Data normalization + feature engineering 3. Seed/anchor resolution from UI 4. Recommendation scoring 5. Duration-constrained playlist selection 6. Playback + debug rendering

Primary files: - `recommendation_app/offline-v2/code/application_v2.py` - `recommendation_app/offline-v2/code/data_upgrade_v2.py` - `recommendation_app/offline-v2/code/recommender_v2_core.py` - `recommendation_app/offline-v2/code/recommender_v2_adapter.py` - `recommendation_app/offline-v2/code/youtube_live_resolver.py`

3. Step-by-Step Implementation

Step 1: Load data

Input options: 1. Kaggle baseline id: salvatorerastelli/spotify-and-youtube 2. Kaggle expansion id: solomonameh/spotify-music-dataset 3. Uploaded CSV fallback

Main path: - read_source_v2(...) in application_v2.py - load_and_merge_from_kagglehub(...) in data_upgrade_v2.py

Step 2: Normalize schema and validate required columns

In prepare_music_data(...): 1. Lower-case all column names 2. Alias streams -> stream (if present) 3. Validate required fields 4. Coerce numeric columns 5. Remove rows with empty artist/track

Required numeric inputs include: - danceability, energy, acousticness, instrumentalness, liveness, valence, tempo - views, likes, comments, stream

Step 3: Engineer core features

Definitions:

1. engagement_rate

$$\text{engagement_rate} = \frac{\text{likes} + \text{comments}}{\text{views}}$$

Implementation detail:

- division by zero is converted to NaN, then filled with 0

2. stream_to_view_ratio

$$\text{stream_to_view_ratio} = \frac{\text{stream}}{\text{views}}$$

Same zero-view handling as above.

3. Min-max normalization helper:

$$\text{norm}(x_i) = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

If max == min, output 0 for all rows.

4. Derived normalized columns:

- views_norm = norm(log(1 + views))
- stream_norm = norm(log(1 + stream))
- engagement_norm = norm(engagement_rate)
- ratio_norm = norm(stream_to_view_ratio)
- tempo_scaled = norm(tempo)
- popularity_norm = norm(log(1 + views + stream))

5. momentum_score

$$\text{momentum_score} = \frac{\text{views_norm} + \text{engagement_norm} + \text{ratio_norm}}{3}$$

Step 4: Link quality + canonical links

Link construction: - Spotify: prioritize spotify:track:... URI, fallback to track search URL - YouTube: use direct video URL only when title match is good enough; otherwise fallback to search query

Improved featured-track matching: - strip feat. / ft. segments for core-title comparison - include featured-artist title hits

Link quality score:

$$\text{link_quality} = \text{spotify_id_present} + \text{youtube_id_present} + \text{youtube_title_match}$$

Step 5: Canonical dedupe (collab guardrail)

To avoid duplicate rows for one collab track with different artist owners:

Canonical song key priority: 1. sp:<spotify_track_id> 2. yt:<youtube_video_id> 3. fb:<clean_artist>::<clean_track>

Sorting preference before dedupe: 1. keep owner rows before rows where listed artist appears only in feature list 2. higher link quality 3. higher momentum/views/stream

Then dedupe by: 1. canonical_song_key 2. fallback artist + track

Step 6: Resolve UI seeds/anchors

Two top-level modes: 1. Quick Mode 2. Self Mix

Self Mix supports: - Pick Songs - Pick Artists - Surprise Me

Selection guardrails: - max 5 combined picks (songs + artists) - Final Pick Box is source of truth

Artist seed resolution: - if user picks an artist, best seed track is selected by highest momentum/views/stream - featured-artist matches are also considered for seed candidate resolution

Step 7: Build preferred artist weights

From current selected state: 1. each selected artist adds +1 weight 2. each selected song adds +1 to its owning artist 3. normalize to sum=1

If no artist signal exists, preferred artist weights = {} .

Step 8: Compute recommendation scores

Feature vector columns: -

danceability, energy, acousticness, instrumentality, liveness, valence, speechiness, tempo_scaled

1. Z-score each feature:

$$z = \frac{x - \mu}{\sigma}$$

If $\sigma=0$, replace with 1 to avoid division error.

2. Build seed target vector:

$$\text{target} = \sum_{s \in \text{seeds}} w_s \cdot z_s$$

where w_s are normalized seed weights.

3. Apply mood adjustment deltas (feature offsets).

4. Similarity score:

$$\cos = \frac{\text{dot}(z_i, \text{target})}{\|z_i\| \cdot \|\text{target}\|}$$
$$\text{similarity_score} = \frac{\cos + 1}{2}$$

5. Platform score:

$$\text{platform_score} = \text{spotify_weight} \cdot \text{stream_norm} + (1 - \text{spotify_weight}) \cdot \text{views_norm}$$

6. Discovery score:

$$\text{discovery_score} = (1 - \text{discovery_mode}) \cdot \text{popularity_norm} + \text{discovery_mode} \cdot (1 - \text{popularity_norm})$$

Interpretation:

- discovery_mode=0 -> hits
- discovery_mode=1 -> hidden gems

7. Hits/hidden helper:

$$\text{hits_preference} = 1 - \text{discovery_mode}$$

8. Bonus terms:

- Seed bonus:

$$\text{seed_bonus} = I(\text{seed}) \cdot (0.02 + 0.26 \cdot \text{hits_preference})$$

- Artist direct bonus:

$$\text{artist_hits_bonus} = \text{artist_weight} \cdot (0.12 \cdot \text{hits_preference})$$

- Hidden-gem proportional bonus:

$$\text{artist_hidden_bonus} = \text{artist_weight} \cdot \text{discovery_mode} \cdot (1 - \text{popularity_norm}) \cdot (0.08 + 0.22 \cdot \text{similarity_score})$$

$$\text{artist_bonus} = \text{artist_hits_bonus} + \text{artist_hidden_bonus}$$

9. Final recommendation score:

$$\text{recommendation_score} = 0.58 \cdot \text{similarity_score} + 0.16 \cdot \text{platform_score} + 0.16 \cdot \text{momentum_score} + 0.10 \cdot \text{discovery_score} + \text{seed_bonus} + ;$$

10. Seed inclusion rule:

- Seeds are excluded by default
- Seeds are included only when UI is 100% Hits

Step 9: Build duration-constrained playlist

Function: `build_duration_playlist(...)`

Inputs: - ranked candidates - `target_minutes` - `tolerance_minutes` (default 3) - `candidate_limit` - `max_tracks`

Guardrail clamps: - `candidate_limit` in [1, 500] - `max_tracks` in [1, 120] - `target_minutes` ≥ 1 - `tolerance_minutes` ≥ 0

Process: 1. Convert/clean durations (fallback to median if missing/zero) 2. Dynamic programming over reachable duration sums: - state: `total_seconds` $\rightarrow$ (`score_sum`, `picked_indexes`) 3. Keep best score per duration sum 4. Choose best playlist: - in tolerance window if possible - else closest total duration - tiebreak by higher score sum

4. Metrics shown in UI/debug view

Each row can expose: - `recommendation_score` - `similarity_score` - `platform_score` - `discovery_score` - `momentum_score` - `views`, `stream`

Interpretation: - high `similarity_score`: acoustically close to seed target - high `platform_score`: aligned with Spotify/YouTube bias - high `discovery_score`: aligned with hits/hidden setting - high `momentum_score`: stronger current engagement signal

5. Quick Top logic (seed shortcuts)

In `application_v2.py`:

1. Quick top songs:

- sort by `momentum_score`, then `views`, then `stream` (descending)
- take top N

2. Quick top artists:

- aggregate per artist:
 - `sum(stream)`, `sum(views)`, `mean(momentum_score)`
- score:

$$\text{artist_score} = \log(1 + \text{artist_streams}) + \log(1 + \text{artist_views}) + 2 \cdot \text{artist_momentum}$$

- sort descending and take top N

6. Edge-case guardrails checklist

Implemented guardrails include: 1. Missing required columns $\rightarrow$ explicit `ValueError` 2. Invalid/non-existent seed $\rightarrow$ explicit `ValueError` 3. NaN-safe score inputs before ranking 4. `top_k` ≤ 0 returns empty result 5. Missing `recommendation_score` in playlist input $\rightarrow$ fallback to momentum 6. Zero/invalid durations $\rightarrow$ median fallback 7. Collab dedupe to single canonical track row 8. Improved YouTube matching for featured tracks 9. Ranked-candidate dedupe by non-empty `spotify_track_id` and `youtube_video_id`

7. Performance notes

Current runtime strategy: 1. Cache data load/preparation 2. Cache recommendation and playlist computation in app layer 3. Restrict playlist optimization to top candidate window (default 180)

Practical behavior: - real dataset remains smooth on local laptop-class runtime - scoring is vectorized in pandas/numpy

8. Developer test commands

Use these before merging changes:

```
python3 -m py_compile recommendation_app/offline-v2/code/*.py
streamlit run recommendation_app/offline-v2/code/application_v2.py --server.port 8504 --server.fileWatcherType no
ne
```

Suggested validation scenarios: 1. 100% Hits + selected favorites -> favorites can appear 2. 100% Hidden Gems + selected artists -> less-popular same-artist tracks appear 3. Collab track in dataset -> only one canonical row in playlist 4. YouTube link on feat. tracks -> direct watch URL preferred when title matches